The Multimedia Player's Assistant

Charleston Southern University Computer Science Senior Project
COIN 499

Background

The entire audio and video world is quickly transitioning to the digital era. Instead of buying and relying solely on Compact Disc (CD) Audio, the everyday consumer is now relying on compressed audio and video by the Motion Picture Experts Group (MPEG). Currently, people use Personal Digital Assistants (PDA), iPods, and other MP3 Portable Audio Players, which now almost entirely make ancient devices such as the Walkman Portable Tape Player and the Discman Portable CD Player obsolete. Although these devices provide the everyday user with mobile audio, most consumers and businesses use a multimedia player on their personal desktop or laptop computer.

The most commonly used operating system for consumers and businesses is the Microsoft Windows Operating System. The most popular Multimedia Player that accompanies this operating system is the Windows Media Player (WMP). WMP allows the user to playback various multimedia files, which includes a variety of audio and video files (MP3, AVI, etc.) and has additional features that allow the user to modify the playback options. WMP does have several competitors that have features that WMP either does not possess or does not implement well.

Although competitors offer the same basic features as the Windows Media Player, most of the features, mainly gimmicks and more user-friendly components, are provided by either a one-time or subscription fee. Conversly, WMP is provided free to anyone who uses the Windows Operating System. WMP has an ActiveX component that provides any web-designer or programmer the opportunity to design an application or system suitable to their own needs. When implemented successfully, one can provide the additional capabilities similar to WMP's competitors without paying for the enhancements.

Problem Statement

The general issue surrounding the Windows Media Player is that even though it's free and allows users to listen to or watch any digital media file, its capabilities do not provide an easy-to-use interface. Some competitors not only charge for their players, but also include numerous gimmicks that are either not very useful or use up a considerable percentage of the Central Processing Unit's speed.

For a better understanding of what to include the program, an online survey provided to a small sample population consisting of students and faculty gave the programmer an idea of what to implement in the program. Some of the features suggested include:

- Easy organization of all media into one library.
- Playback of more than one audio/video track
- Blended transitioning (Crossfading)

Objectives

The primary objective of The Multimedia Player's Assistant is to implement the WMP ActiveX Component in a simple program. In addition to the basic controls of the WMP ActiveX Component (Play, Stop, etc.), the program will also allow the controller to accomplish the following:

- Adding, deleting, or modifying tracks within a simple data table (Library). This data table can be modified using Microsoft Access or, if the user prefers, modifications as well as searches can and should be made within a simple data form already implemented within the program in a user-friendly interface. This data form also provides the opportunity to open and preview a track as well as optionally add the title and the length of the track automatically. This table contains the following entities [Note: Although the entities listed below are in the data table, not all entities will be visible in the main interface to the user for record safety purposes except when making changes]:
    - Track Number (An Automatic Counter)
    - Track Name (A String field containing the title of the track)
    - File Name (A String field containing the location of the track)
    - Artist (A String field containing the artist/composer of the track)
    - Duration (A String field containing the length of the track)
    - Genre (A String field containing a user-specified genre of the track)
    - Image File (An optional String field that contains an image file [and its location] to be displayed in an image box while the track is playing)
- A simple user-defined playlist run in either ordered or random playback (User-specified).
- Dual instances of the WMP ActiveX Component. This add-on provides the user with two possibilities:
    - Simultaneous or separate playback of two tracks.
    - Blended transitioning from one track to another either:
        - Automatically (Player waits until track nears completion within 1, 5, or 10 seconds [User-specified] and proceeds with the transitioning)
        - Manually (User activates the transition by simple click)
- Adjustment of the display of the interface by selecting one of three presets:
    - Normal without Library (Library not visible)
    - Normal with Library (Library and it's controls will be displayed below the normal interface)
    - Compact (Only controls for both ActiveX instances are visible and the interface size is reduced to less than ¼ of the screen size [Based on 1024 x 768 screen resolution])
- Features from Windows Media Player Version 10 including:
    - Visualizations
    - Volume Control
    - Rewind/Fast-Forward
    - Option for Full-Screen Display

Assumptions

One assumption of this program is that the player can only work with certain audio/video files within the WMP's specifications including:

- All Audio Files (.mp3, .wav, .snd, .au, .aif, .aifc, .aiff, .wma, .mid, .rmi, .midi)
- All Video Files (.avi, .mpeg, .mpg, .m1v, .mp2, .mpa, .mpe, .ifo, .vob, .wmv)
- CD Audio Tracks/Files (.cda)

Reasons for this assumption are that certain files (RealMedia and WinAmp) can only be legally run through their associated programs. Secondly, the user may not always provide each track with an associated image. That being the case, a default image file will be substituted. It can also be assumed that the user will/must have a Windows Operating System that has the .NET framework, which can be downloaded for free from Microsoft's online site.

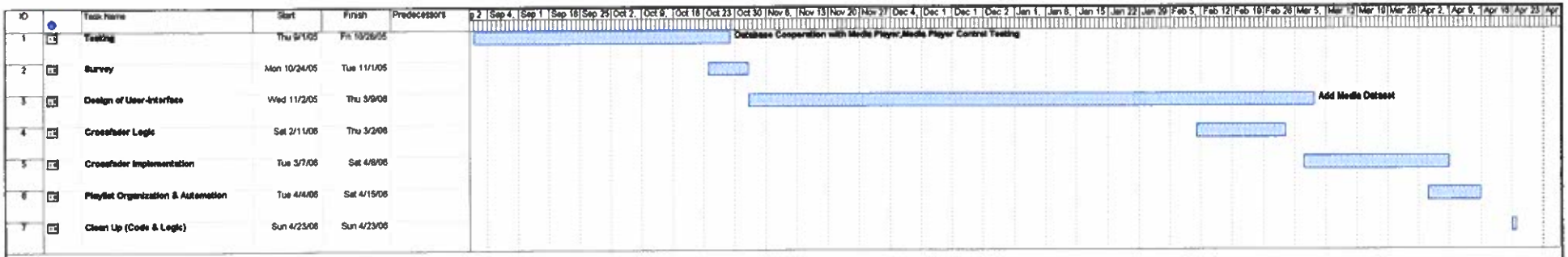
System Processes

This program (and its database) was developed in the Visual Basic.NET programming language and Access SQL (Using a Microsoft Jet OLE Database Adapter) in order to provide the programmer with less limitations on implementing certain features (i.e. WMP ActiveX Component, Access MDB files) and to ensure user-friendliness. All access to the Multimedia Player's Assistant is unlimited to the end user of the program. The user will be able to make the necessary changes to the Library data table including adding or deleting tracks and their associated fields.

Project Scope

This program is designed for all types of users, leisure or business. Boundaries include, but are not limited to, WMP-compatible files, a minimum requirement of 32 MB of RAM, a Windows Operating System with .NET Framework, at least 2MB of hard disk space (for program executable, media library database file, and ActiveX files), and CPU speed of at least 350 MHz.

Although version 1.0 of the Multimedia Player's Assistant is slated for completion by the end of April 2006, updates and enhancements to this program are certainly not to be ruled out. For example, Visual Basic.NET also provides the programmer to design applications for PDA's that run Windows Compact Edition (CE). It is possible that this project can be implemented in this environment, hence possibly competing in the market of portable use of multimedia files. Although the initial design and implementation of the Multimedia Player's Assistant is designated for use on a personal computer (PC), updating and enhancements could eventually bring the program in good competition in the multimedia industry.


ID
Task Name
Start
Finish
Predecessors
1 Testing Thu 9/1/05 Fri 10/28/05
2 Survey Mon 10/24/05 Tue 11/1/05
3 Design of User-Interface Wed 11/2/05 Thu 3/9/06
4 Crossfader Logic Sat 2/11/06 Thu 3/2/06
5 Crossfader Implementation Tue 3/7/06 Sat 4/8/06
6 Playlist Organization & Automation Tue 4/4/06 Sat 4/15/06
7 Clean Up (Code & Logic) Sun 4/23/06 Sun 4/23/06
Database Cooperation with Media Player,Media Player Control Testing
Add Media Dataset

Question	Answer
Compatibility for MP3, WAV, MPG, AVI, WMV, WMA, and various other multimedia formats.	Very Important
Database/Playlist to keep organization of track name, file location, artist, duration, etc.	Important
Dual multimedia players, allow user to listen to two tracks separately or simultaneously.	Not Important
Crossfader that fades out ending audio and fades in the next audio track.	Important
Image display to accompany associated audio track.	Undecided
User-specified playback of tracks by playlist (Ordered, Random, Continuous)	Important
Customizable interface that allows user to 'decorate' their player with solid colors or establishing a background image.	Undecided
Please add any suggestions for additional features that you would be interested in a media player.	<Unanswered>
Compatibility for MP3, WAV, MPG, AVI, WMV, WMA, and various other multimedia formats.	Important
Database/Playlist to keep organization of track name, file location, artist, duration, etc.	Very Important
Dual multimedia players, allow user to listen to two tracks separately or simultaneously.	Not Important
Crossfader that fades out ending audio and fades in the next audio track.	Important
Image display to accompany associated audio track.	Undecided
User-specified playback of tracks by playlist (Ordered, Random, Continuous)	Very Important
Customizable interface that allows user to 'decorate' their player with solid colors or establishing a background image.	Not Important
Please add any suggestions for additional features that you would be interested in a media player.	None-but good luck!!!!!! =)
Compatibility for MP3, WAV, MPG, AVI, WMV, WMA, and various other multimedia formats.	Important
Database/Playlist to keep organization of track name, file location, artist, duration, etc.	Important
Dual multimedia players, allow user to listen to two tracks separately or simultaneously.	Undecided
Crossfader that fades out ending audio and fades in the next audio track.	Not Important
Image display to accompany associated audio track.	Not Important
User-specified playback of tracks by playlist (Ordered, Random, Continuous)	Important
Customizable interface that allows user to 'decorate' their player with solid colors or establishing a background image.	Not Important
Please add any suggestions for additional features that you would be interested in a media player.	<Unanswered>
Compatibility for MP3, WAV, MPG, AVI, WMV, WMA, and various other multimedia formats.	Important
Database/Playlist to keep organization of track name, file location, artist, duration, etc.	Very Important
Dual multimedia players, allow user to listen to two tracks separately or simultaneously.	Not Important
Crossfader that fades out ending audio and fades in the next audio track.	Important
Image display to accompany associated audio track.	Important
User-specified playback of tracks by playlist (Ordered, Random, Continuous)	Very Important
Customizable interface that allows user to 'decorate' their player with solid colors or establishing a background image.	Important
Please add any suggestions for additional features that you would be interested in a media player.	<Unanswered>
Compatibility for MP3, WAV, MPG, AVI, WMV, WMA, and various other multimedia formats.	Very Important
Database/Playlist to keep organization of track name, file location, artist, duration, etc.	Very Important
Dual multimedia players, allow user to listen to two tracks separately or simultaneously.	Not Important
Crossfader that fades out ending audio and fades in the next audio track.	Not Important
Image display to accompany associated audio track.	Important
User-specified playback of tracks by playlist (Ordered, Random, Continuous)	Important
Customizable interface that allows user to 'decorate' their player with solid colors or establishing a background image.	Undecided
Please add any suggestions for additional features that you would be interested in a media player.	<Unanswered>
Compatibility for MP3, WAV, MPG, AVI, WMV, WMA, and various other multimedia formats.	Very Important
Database/Playlist to keep organization of track name, file location, artist, duration, etc.	Very Important
Dual multimedia players, allow user to listen to two tracks separately or simultaneously.	Undecided
Crossfader that fades out ending audio and fades in the next audio track.	Important
Image display to accompany associated audio track.	Important
User-specified playback of tracks by playlist (Ordered, Random, Continuous)	Very Important
Customizable interface that allows user to 'decorate' their player with solid colors or establishing a background image.	Important
Please add any suggestions for additional features that you would be interested in a media player.	What about playing streaming audio over the internet?
	Could you get it to play songs backwards to look for hidden messages?
	Could you create a copy/paste feature to copy parts of songs as sound clips?

Survey conducted 10/27/2005

Source Code For The Multimedia Player's Assistant
Form1.vb (Main Form)

```
'The Multimedia Player's Assistant
'James B. Openshaw
'Charleston Southern University COIN Senior Project
Public Class Form1 'This is the main form
    Inherits System.Windows.Forms.Form 'Form inherits properites from the system
 Windows Form Designer generated code
    'Declared Global Variables
    'These variables are global mainly because they will be used in more than one function
    Dim CDAudio As Boolean = False 'If there are CD tracks on the playlist
    Dim TrackA, TrackB As String 'Used to display name of track on the player
    Dim DeckA As Boolean = False 'Indicates if PlayerA is running
    Dim DeckB As Boolean = False 'Indicates if PlayerB is running
    Dim ALoad As Boolean = False 'Indicates if PlayerA has a track loaded on it
    Dim BLoad As Boolean = False 'Indicates if PlayerB has a track loaded on it
    Dim Acrossed As Boolean = False 'Indicates if playerA was faded out
    Dim Bcrossed As Boolean = False 'Indicates if playerB was faded out
    Dim listloop As Boolean = False 'Used to check if playlist will loop
    Dim playlistNum As Integer = 1 'Used for playlist order
    Dim listrun As Integer = 1 'Used for playlist run
    Dim ImageA, ImageB As String 'Used for image order
    Structure PlayLst 'Structure for playlist control
        Dim TrackName As String 'Used for Track Name display purposes only
        Dim FileName As String 'Used for file referencing for track
        Dim ImgFile As String 'Used for Image Associated with file
    End Structure 'Structure ends here
    Dim list(100) As PlayLst 'No playlist should have any more than 100 tracks

    'BtnUpdateDB allows user to make any changes to the library data set
    Private Sub BtnUpdateDB_Click(ByVal sender As System.Object, ByVal e As System.
    EventArgs) Handles BtnUpdateDB.Click
        MenuShowLib.PerformClick() 'Closes and hides the Datagrid when updating Library
        Dim libfrm As New LibraryForm 'Must declare the dataform before opening
        libfrm.Show() 'Displays the dataform add/changing library data

    End Sub

    'Form1_Load prepares the Library Data Set and basically
    'sets all default parameters for the media players
    Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles MyBase.Load
        OleDbDataAdapter1.Fill(DataSet11) 'Datagrid is filled with library data
        DataGrid1.Visible = False 'Keeps the datagrid invisible until requested
        Width = 798 'Sets default window width
        Height = 480 'Sets default window length
        MediaPlayerA.settings.autoStart = False 'Makes sure PlayerA doesn't automatically
    start when a file is loaded
        MediaPlayerB.settings.autoStart = False 'Makes sure PlayerB doesn't automatically
    start when a file is loaded
        showpic("default.jpg") 'Has the default image displayed on the picture box
        InitializePlayers() 'Gets player A and B set up
        resetList() 'Makes sure playlist is reset
    End Sub

    'This function simply opens a file for playerA
    Private Sub BtnOpenA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles BtnOpenA.Click
        OpenFileDialog1.FileName = MediaPlayerA.URL 'sets filename in openfiledialog box
        OpenFileDialog1.ShowDialog() 'Opens OpenFileDialog Box
        loadA(OpenFileDialog1.FileName) 'Loads selected file into player
        ImageA = "C:\MPA\default.jpg" 'Sets imageA to default image
    End Sub

    'This function will play the file and set
    'certain flags for informational display and additional functions
    Private Sub BtnPlayA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles BtnPlayA.Click
```

```
        MediaPlayerA.Ctlcontrols.play() 'Plays PlayerA
        If ChkVisual.Checked Then 'Checks if visualizations were enabled, if so...
            MediaPlayerB.Visible = False 'Visualizations for PlayerB are hidden
            MediaPlayerA.Visible = True 'Visualizations for PlayerA are diplayed
        End If
        BtnStopA.Enabled = True 'Stop button for PlayerA enabled
        showpic(ImageA) 'Calls function to load image a into a picture box
        DeckA = True 'PlayerA is running
        DeckB = False 'PlayerB is not running
        BtnOpenA.Enabled = False
        TimerA.Start() 'Timer to monitor PlayerA is activated
        LblTitleA.Text = MediaPlayerA.currentMedia.name 'Places Track Name in PlayerA's box

    End Sub

    'This function will stop the file and reset
    'certain parameters
    Private Sub BtnStopA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles BtnStopA.Click
        MediaPlayerA.Ctlcontrols.stop() 'Stops PlayerA
        BtnOpenA.Enabled = True 'Enables file opener for playerA
        DeckA = False 'PlayerA is not running
        TimerA.Stop() 'TimerA is deactivated
        LblRemainA.Text = "00:00/" & MediaPlayerA.currentMedia.durationString 'Displays ↙
    time progress of track and compares it with actual length
    End Sub

    'This function simply opens the file for PlayerB
    Private Sub BtnOpenB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles BtnOpenB.Click
        OpenFileDialog1.FileName = MediaPlayerB.URL 'Sets filename on PlayerB to ↙
    OpenFileDialog1
        OpenFileDialog1.ShowDialog() 'Displays OpenFileDialog Box
        loadB(OpenFileDialog1.FileName) 'Calls function to load file into playerB
        ImageB = "C:\MPA\default.jpg" 'Sets ImageB to the default image
    End Sub

    'This function increases the volume for PlayerA
    Private Sub BtnVolUpA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)↙
     Handles BtnVolUpA.Click
        MediaPlayerA.settings.volume += 10 'Increases PlayerA volume by 10 units
        BtnVolDownA.Visible = True 'Displays button to lower volume for PlayerA
        If MediaPlayerA.settings.volume = 100 Then 'Checks to see if volume is at maximum, ↙
    if so...
            BtnVolUpA.Visible = False 'Button to increase volume disappears
        End If
        crossCheck() 'Runs a check on the volume for crossfading
    End Sub

    'This function decrease the volume for PlayerA
    Private Sub BtnVolDownA_Click(ByVal sender As System.Object, ByVal e As System. ↙
    EventArgs) Handles BtnVolDownA.Click
        MediaPlayerA.settings.volume -= 10 'Reduces PlayerA volume by 10 units
        BtnVolUpA.Visible = True 'Displays button to increase volume for PlayerA
        If MediaPlayerA.settings.volume = 0 Then 'Checks to see if volume is at minimum, if↙
     so...
            BtnVolDownA.Visible = False 'Button to decrease volume disappears
        End If
        crossCheck() 'Runs a check on the volume for crossfading
    End Sub

    'This function activates visualizations and video viewing for both players
    Private Sub ChkVisual_CheckedChanged(ByVal sender As System.Object, ByVal e As System. ↙
    EventArgs) Handles ChkVisual.CheckedChanged
        If ChkVisual.Checked Then 'Checks to see if Visualizations are to be displayed, if ↙
    so...
```

```
            If DeckA Then 'Checks to see if PlayerA is running, if so...
                MediaPlayerA.Visible = True 'Visualization for PlayerA is shown
            Else
                MediaPlayerB.Visible = True 'Visualization for PlayerB is shown
            End If 'End of If Statement
        Else
            MediaPlayerA.Visible = False 'Visualization for PlayerA disappears
            MediaPlayerB.Visible = False 'Visualization for PlayerB disappears
        End If 'End of if statement
    End Sub

    'This function "rewinds" the media file back 10 seconds (PlayerA)
    Private Sub BtnBackA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles BtnBackA.Click
        MediaPlayerA.Ctlcontrols.currentPosition -= 10 'Current Position of track jumps ↙
    back 10 seconds
        LblRemainA.Text = MediaPlayerA.Ctlcontrols.currentPositionString & "/" & ↙
    MediaPlayerA.currentMedia.durationString 'Updates duration of track
    End Sub

    'This function "fast-forwards" the media file by 10 seconds (PlayerA)
    Private Sub BtnForwardA_Click(ByVal sender As System.Object, ByVal e As System. ↙
    EventArgs) Handles BtnForwardA.Click
        MediaPlayerA.Ctlcontrols.currentPosition += 10 'Current Position for PlayerA jumps ↙
    forward by 10 seconds
        LblRemainA.Text = MediaPlayerA.Ctlcontrols.currentPositionString & "/" & ↙
    MediaPlayerA.currentMedia.durationString 'Updates duration of track
    End Sub

    'This Timer function controls all the progressive functions for PlayerA
    'Including Timing and reseting
    Private Sub TimerA_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles TimerA.Tick
        LblRemainA.Text = MediaPlayerA.Ctlcontrols.currentPositionString & "/" & ↙
    MediaPlayerA.currentMedia.durationString 'Updates duration of track
        If MediaPlayerA.Ctlcontrols.currentPosition = 0 Then  'Checks to see if Player is ↙
    isn't running/stopped
            BtnStopA.PerformClick() 'Stops the track
        ElseIf ChkCross.Checked And DeckA And BLoad And (CrossTimer.Enabled = False) And ( ↙
    (CInt(MediaPlayerA.currentMedia.duration) - CInt(MediaPlayerA.Ctlcontrols. ↙
    currentPosition)) <= CInt(CmbCross.Text)) Then 'Checks for position in order to ↙
    crossfade
            ALoad = False 'playerA needs another file
            BtnCrossNow.PerformClick() 'Activates the crossfader
        End If
    End Sub

    'This function allows PlayerB to play
    Private Sub BtnPlayB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles BtnPlayB.Click
        MediaPlayerB.Ctlcontrols.play() 'PlayerB plays
        If ChkVisual.Checked Then 'Checks to see if visualizations are enabled, if so...
            MediaPlayerA.Visible = False 'Visualization for PlayerA diappears
            MediaPlayerB.Visible = True 'Visualization for PlayerB appears
        End If
        showpic(ImageB) 'Loads ImageB into picture box
        BtnStopB.Enabled = True 'Stop button for PlayerB is enabled
        BtnOpenB.Enabled = False 'Enables button to open a file for playerB
        DeckB = True 'PlayerB is running
        DeckA = False 'PlayerA is not running
        TimerB.Start() 'Timer to monitor PlayerB is activated
        LblTitleB.Text = MediaPlayerB.currentMedia.name 'Track Name for PlayerB is ↙
    displayed

    End Sub
```

```
    'This function rewinds PlayerB by 10 seconds
    Private Sub BtnBackB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles BtnBackB.Click
        MediaPlayerB.Ctlcontrols.currentPosition -= 10 'Track jumps back 10 seconds
        LblRemainB.Text = MediaPlayerB.Ctlcontrols.currentPositionString & "/" &
    MediaPlayerB.currentMedia.durationString 'Updates duration of the track
    End Sub

    'This function pauses PlayerA
    Private Sub BtnPauseA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles BtnPauseA.Click
        MediaPlayerA.Ctlcontrols.pause() 'PlayerA pauses
        BtnPlayA.Enabled = True 'Button to play PlayerA is enabled
        TimerA.Stop() 'Timer to monitor PlayerA is disabled
    End Sub

    'This function pauses PlayerB
    Private Sub BtnPauseB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles BtnPauseB.Click
        MediaPlayerB.Ctlcontrols.pause() 'PlayerB pauses
        BtnPlayB.Enabled = True 'Button to play PlayerB is enabled
        TimerB.Stop() 'Timer to monitor PlayerB is disabled
    End Sub

    'This function fast-forwards the track for PlayerB by 10 seconds
    Private Sub BtnForwardB_Click(ByVal sender As System.Object, ByVal e As System.
    EventArgs) Handles BtnForwardB.Click
        MediaPlayerB.Ctlcontrols.currentPosition += 10 'Track playing in PlayerB jumps
    forward by 10 seconds
        LblRemainB.Text = MediaPlayerB.Ctlcontrols.currentPositionString & "/" &
    MediaPlayerB.currentMedia.durationString 'Updates duration of the track
    End Sub

    'This function Stops the playing track in PlayerB
    Private Sub BtnStopB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles BtnStopB.Click
        MediaPlayerB.Ctlcontrols.stop() 'Stops PlayerB
        BtnOpenB.Enabled = True 'Enables button to open another track for playerB
        TimerB.Stop() 'Timer to monitor PlayerB is disabled
        LblRemainB.Text = "00:00/" & MediaPlayerB.currentMedia.durationString 'Progress
    timer for for PlayerB is set to 0
    End Sub

    'This function loads item selected in library into PlayerA and displays its image, if
    any
    Private Sub BtnRunA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles BtnRunA.Click
        Dim i As Integer = DataGrid1.CurrentRowIndex 'returns an integer for the selected
    row
        Dim fileName As String = DataSet11.Tables(0).Rows(i).Item("FileName") 'Grabs
    filename string from row selected
        loadA(fileName) 'Loads selected file into playerA
        Try
            ImageA = DataSet11.Tables(0).Rows(i).Item("TrackPic") 'Loads image file string
    into ImageA
        Catch
            'Do nothing, prevents an exception crash
        End Try

    End Sub

    'This function loads item selected library into PlayerB and displays its image, if any
    Private Sub BtnRunB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles BtnRunB.Click
        Dim i As Integer = DataGrid1.CurrentRowIndex 'Returns an integer for the selected
    row
```

```
    Dim fileName As String = DataSet11.Tables(0).Rows(i).Item("FileName") 'Grabs
filename string from row selected
    loadB(fileName) 'Loads selected file into playerB
    Try
        ImageB = DataSet11.Tables(0).Rows(i).Item("TrackPic") 'Loads image file string
into ImageB
    Catch
        'Do nothing, prevents an exception crash
    End Try
End Sub

'This function increases the volume in PlayerB by 10 units
Private Sub BtnVolUpB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
 Handles BtnVolUpB.Click
    MediaPlayerB.settings.volume += 10 'Increases volume for PlayerB by 10 units
    BtnVolDownB.Visible = True 'Enables button to reduce volume for PlayerB
    If MediaPlayerB.settings.volume = 100 Then 'Checks to see if volume for PlayerA is
at maximum, if so...
        BtnVolUpB.Visible = False 'Button for increasing volume for PlayerB is disabled
    End If
    crossCheck() 'calls function to modify crossfader
End Sub

'This function reduces the volume in PlayerB by 10 units
Private Sub BtnVolDownB_Click(ByVal sender As System.Object, ByVal e As System.
EventArgs) Handles BtnVolDownB.Click
    MediaPlayerB.settings.volume -= 10 'Reduces volume for PlayerB by 10 units
    BtnVolUpB.Visible = True 'Enables button to increase volume for PlayerB
    If MediaPlayerB.settings.volume = 0 Then 'Checks to see if volume for PlayerB is at
 minimum, if so...
        BtnVolDownB.Visible = False 'Button for decreasing volume for PlayerB is
disabled
    End If
    crossCheck() 'calls function to modify crossfader
End Sub

'This function displays an image associated with the track, if any
Private Sub showpic(ByVal Pic As String)
    Try
        MPPicBox.Image = Image.FromFile(Pic) 'loads image into the Picture Box
    Catch
        'Do Nothing, prevents an exception crash
    End Try
End Sub

'This function monitors the progress of PlayerB
Private Sub TimerB_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles TimerB.Tick
    LblRemainB.Text = MediaPlayerB.Ctlcontrols.currentPositionString & "/" &
MediaPlayerB.currentMedia.durationString 'Updates duration of track
    If MediaPlayerB.Ctlcontrols.currentPosition = 0 Then  'Checks to see if PlayerB has
 stopped, if so...
        BtnStopB.PerformClick() 'Stop button for PlayerB is clicked
    ElseIf ChkCross.Checked And DeckB And ALoad And (CrossTimer.Enabled = False) And (
(CInt(MediaPlayerB.currentMedia.duration) - CInt(MediaPlayerB.Ctlcontrols.
currentPosition)) <= CInt(CmbCross.Text)) Then 'Checks conditions for a crossfade
        BLoad = False 'PlayerB ready for next track
        BtnCrossNow.PerformClick() 'Activates crossfader
    End If
End Sub

'This function adds a track in PlayerA to playlist
Private Sub BtnAddA_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles BtnAddA.Click
    OpenFileDialog1.ShowDialog() 'Opens OpenFileDialog Box
    Dim trackName As String = OpenFileDialog1.FileName 'Sets chosen file name to
```

```
    PlayerA
        Dim fileName As String = trackName 'sets filename as trackname
        list(playlistNum).TrackName = trackName 'sets trackname in structure
        list(playlistNum).FileName = fileName 'sets filename in structure
        With LstPlaylist.Items
            .Add(playlistNum & "-" & list(playlistNum).TrackName) 'Adds trackname and
    number to playlist
        End With
        If list(playlistNum).FileName.EndsWith("cda") Then
            CDAudio = True
            ChkCross.Checked = False
            ChkCross.Enabled = False
        End If
        BtnRunPlaylist.Enabled = True 'Enables ability to run playlist
        playlistNum += 1 'increments playlist
        CmbPlayback.Enabled = True 'enables playback controller
    End Sub

    'This function will run the playlist
    Private Sub BtnRunPlaylist_Click(ByVal sender As System.Object, ByVal e As System.
    EventArgs) Handles BtnRunPlaylist.Click
        BtnRunA.Enabled = False 'Disables track loading from library to playerA
        BtnRunB.Enabled = False 'Disables track loading from library to playerB
        If CDAudio Then
            runCDA()
        Else
            ChkCross.Checked = True 'enables crossfader
            ChkCross.Visible = False 'removes checkbox for crossfader
            Dim R As New Random 'sets up a random number generator
            Select Case CmbPlayback.Text 'Condition for playback
                Case Is = "Ordered"
                    loadA(list(listrun).FileName) 'loads file form structure into playerA
                    ImageA = (list(listrun).ImgFile) 'loads image file into imageA
                    If list(listrun + 1).TrackName <> "Empty" Then 'Checks if next track is
     empty
                        listrun += 1 'increments listrun
                        loadB(list(listrun).FileName) 'loads file from structure into
    playerB
                        ImageB = (list(listrun).ImgFile) 'loads image file into ImageB
                    End If
                Case Is = "Random"
                    Dim i, j As Integer 'sets up two variables from random numbers
                    i = R.Next(1, playlistNum) 'i is random
                    j = R.Next(1, playlistNum) 'j is random
                    While j = i 'checks to see if random number i matches j
                        j = R.Next(1, playlistNum) 'another number is generated for j
                    End While
                    MediaPlayerA.URL = list(i).FileName 'sets file in playerA
                    ImageA = list(i).ImgFile 'sets image file in playlist to imageA
                    MediaPlayerB.URL = list(j).FileName 'sets file in playerB
                    ImageB = list(j).ImgFile 'sets image file in playlist to imageB
                    'BtnPlayB.PerformClick() 'sets playerB up by running it first
                    'MediaPlayerB.Ctlcontrols.stop() 'stops playerB
                    'BtnStopB.PerformClick()
                Case Is = "Continuous"
                    loadA(list(listrun).FileName) 'loads file form structure into playerA
                    ImageA = (list(listrun).ImgFile) 'loads image file into imageA
                    If list(listrun + 1).TrackName <> "Empty" Then 'Checks if next track is
     empty
                        listrun += 1 'increments listrun
                        loadB(list(listrun).FileName) 'loads file from structure into
    playerB
                        ImageB = (list(listrun).ImgFile) 'loads image file into ImageB
                    End If
            End Select
            BLoad = True 'playerB is ready
```

```
            ALoad = True 'playerA is ready
        End If
        CmbPlayback.Enabled = False 'deactivates playback controls
        BtnPlayA.Enabled = True 'enables playerA play button
        BtnClear.Enabled = False 'Disables button to clear and reset playlist
        PlayTimer.Enabled = True 'Activates timer to monitor playlist run
        BtnRunPlaylist.Enabled = False 'Disables playlist run button
        BtnStopPLST.Enabled = True
        BtnPlayA.PerformClick() 'PlayerA begins
    End Sub

    'This function resets the Playlist Structure
    Private Sub resetList()
        Dim i As Integer 'Starts the list at 1
        For i = 1 To 100 'For the entire list...
            list(i).TrackName = "Empty" 'Reset the track name
            list(i).FileName = "Empty" 'Reset the File Name
            list(i).ImgFile = "C:\MPA\default.jpg" 'Sets default image for all image items ↙
    in the list
        Next
    End Sub

    'This function clears the playlist
    Private Sub BtnClear_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles BtnClear.Click
        resetList() 'Calls the function to reset the playlist
        LstPlaylist.Items.Clear() 'Clears the list display
        playlistNum = 1 'Playlist is ready to start over
        BtnRunPlaylist.Enabled = False 'Disabled button to run the playlist
        CDAudio = False 'List is empty so no CD Audio is in the list
        ChkCross.Enabled = True 'Reenables crossfader check
    End Sub

    'This function simply closes the program
    Private Sub MenuExit_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles MenuExit.Click
        End
    End Sub

    'This function displays/hides the library and its controls
    Private Sub MenuShowLib_Click(ByVal sender As System.Object, ByVal e As System.        ↙
    EventArgs) Handles MenuShowLib.Click
        If MenuShowLib.Checked = False Then 'Checks to see if the library is visible, if   ↙
    not...
            OleDbDataAdapter1.Fill(DataSet11) 'Fill/Update the library list
            DataGrid1.Visible = True 'Show the Data Grid
            BtnUpdateDB.Visible = True 'Show Update button
            BtnRunA.Visible = True 'Show RunA button
            BtnRunB.Visible = True 'Show RunB button
            BtnLoadPlay.Visible = True 'Show Load into Playlist button
            Form1.ActiveForm.Width = 798 'Sets form width to accommodate library
            Form1.ActiveForm.Height = 632 'Sets for height to accommodate library
            Form1.ActiveForm.FormBorderStyle = FormBorderStyle.FixedSingle 'Sets form to    ↙
    non-adjustable property
            MenuShowLib.Checked = True 'Shows that library is displayed
            MenuSize.Enabled = False 'Deactivates adjusting the view of the Form
        Else
            DataGrid1.Visible = False 'Hides DataGrid
            BtnUpdateDB.Visible = False 'Hides Update button
            BtnRunA.Visible = False 'Hides RunA button
            BtnRunB.Visible = False 'Hides RunB button
            BtnLoadPlay.Visible = False 'Hides Load into Playlist button
            If MenuSize.Text = "Normal View" Then 'Checks to see if Form was in compact     ↙
    view, if so...
                Form1.ActiveForm.Width = 400 'sets form width
                Form1.ActiveForm.Height = 288 'sets form height
```

```
            Form1.ActiveForm.FormBorderStyle = FormBorderStyle.FixedToolWindow 'sets
form to smaller non-adjustable property
        Else
            Form1.ActiveForm.Height = 488 'sets form height
        End If
        MenuSize.Enabled = True 'Enables button to reset form size
        MenuShowLib.Checked = False 'Shows that library isn't displayed
    End If
End Sub

'This function controls the view of the application
Private Sub MenuSize_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles MenuSize.Click
    If MenuSize.Text = "Compact View" Then
        Form1.ActiveForm.Width = 400 'Sets form width to 400
        Form1.ActiveForm.Height = 288 'sets form height to 288
        Form1.ActiveForm.FormBorderStyle = FormBorderStyle.FixedToolWindow 'Readjust
form to accompany compact size
        MenuSize.Text = "Normal View" 'Resets menu selection for normal viewing
    Else
        Form1.ActiveForm.Width = 798 'Sets form width to 798
        Form1.ActiveForm.Height = 480 'sets form height to 480
        Form1.ActiveForm.FormBorderStyle = FormBorderStyle.FixedSingle 'Readjust form
to accompany normal size
        MenuSize.Text = "Compact View" 'Resets menu selection for compact viewing
    End If
End Sub
'This function prepares the crossfader
Private Sub ChkCross_CheckedChanged(ByVal sender As System.Object, ByVal e As System.
EventArgs) Handles ChkCross.CheckedChanged
    crossVisible() 'calls function to set visibility for crossfder
    crossCheck() 'calls function to check crossfader set up
End Sub
'This function calls fucntion to set up crossfader
Private Sub CmbCross_SelectedIndexChanged(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles CmbCross.SelectedIndexChanged
    If (CDbl(CmbCross.Text) < 1) Then
        CmbCross.Text = 1 'To prevent a crash in crossfader calculation
    End If
    crossCheck() 'calls function to check crossfader set up
End Sub
'This function runs the crossfader
Private Sub CrossTimer_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs)
 Handles CrossTimer.Tick
    BtnCrossNow.Text = "Crossfading" 'Button is renamed to "Crossfading"
    BtnCrossNow.Enabled = False 'Disables crossfader enabler button
    If (DeckA) Then 'If playerA is currently running...
        BtnVolDownB.Visible = True 'button to decrease volume is visible
        KillControls("A") 'Calls function to disable certain controls for playerA
        TimerA.Enabled = False 'disabled timerA
        MediaPlayerB.Ctlcontrols.play() 'runs playerb
        TimerB.Enabled = True 'enables timerB
        MediaPlayerA.settings.volume -= 1 'Reduces volume in playerA
        MediaPlayerB.settings.volume += 1 'increases volume in playerB
        If (MediaPlayerA.settings.volume = 0) Then 'when volume for playerA is down to
0
            CrossTimer.Enabled = False 'Crossfader stops
            DeckA = False 'playerA is no longer running
            Acrossed = True 'playerA has faded out
            ALoad = False 'playerA is ready for another track
            DeckB = True 'playerB is now running
            BtnBackA.Enabled = True 'enables rewind button for playerA
            BtnForwardA.Enabled = True 'enables fast forward button for playerA
            BtnVolDownA.Visible = False 'disbales button to reduces volume for playerA
            BtnStopA.PerformClick() 'Stops playerA
            LblRemainA.Text = "Stopped" 'states that playerA has stopped
```

```
            BtnOpenA.Enabled = True 'enables ability to op another file for playerA
            BtnCrossNow.Text = "Crossfade Now" 'text for crossfader button is changed
            BtnCrossNow.Enabled = True 'enables crossfader button
        End If
    Else
        BtnVolDownA.Visible = True 'button to reduce volume for playerA is visible
        KillControls("B") 'certain controls for playerB are disabled
        TimerA.Enabled = True 'timer for playerA enabled
        TimerB.Enabled = False 'timer for playerB disabled
        MediaPlayerA.Ctlcontrols.play() 'playerA runs
        MediaPlayerB.settings.volume -= 1 'volume for playerB is reduced by 1
        MediaPlayerA.settings.volume += 1 'volume for playerA is increased by 1
        If (MediaPlayerB.settings.volume = 0) Then 'when volume for playerB is down to
0
            DeckB = False 'playerB is not running
            Bcrossed = True 'playerB has faded out
            BLoad = False 'playerB is ready for next track
            DeckA = True 'playerA is running
            BtnBackB.Enabled = True 'rewind button for playerB is enabled
            BtnForwardB.Enabled = True 'fast-forward button for playerB is enabled
            BtnStopB.PerformClick() 'playerB stops
            LblRemainB.Text = "Stopped" 'indicates that playerB has stopped
            CrossTimer.Enabled = False 'disables crossfader
            BtnVolDownB.Visible = False 'button for volume down for playerB disappears
            BtnOpenB.Enabled = True 'button to open file for playerB is enabled
            TimerB.Enabled = False 'timerB is disabled
            BtnCrossNow.Text = "Crossfade Now" 'crossfader button renamed
            BtnCrossNow.Enabled = True 'crossfader button enabled
        End If
    End If

End Sub
'This function activates the crossfade timer
Private Sub BtnCrossNow_Click(ByVal sender As System.Object, ByVal e As System.
EventArgs) Handles BtnCrossNow.Click
    CrossTimer.Enabled = True 'enables crossfade timer
End Sub
'This function loads a track into the playlist
Private Sub BtnLoadPlay_Click(ByVal sender As System.Object, ByVal e As System.
EventArgs) Handles BtnLoadPlay.Click
    Dim i As Integer = DataGrid1.CurrentRowIndex 'i is what gets selected
    Dim trackName As String = DataSet11.Tables(0).Rows(i).Item("TrackName") 'trackname
is set
    Dim fileName As String = DataSet11.Tables(0).Rows(i).Item("FileName") 'filename is
set
    Dim imagefile As String = DataSet11.Tables(0).Rows(i).Item("TrackPic") 'image is
set
    list(playlistNum).TrackName = trackName 'trackname loaded into playlist
    list(playlistNum).FileName = fileName 'filename loaded into playlist
    list(playlistNum).ImgFile = imagefile 'imagefile loaded into playlist
    CmbPlayback.Enabled = True 'enables playback selection
    With LstPlaylist.Items
        .Add(playlistNum & "-" & list(playlistNum).TrackName) 'adds track to visible
playlist
    End With
    BtnRunPlaylist.Enabled = True 'button to run playlist enabled
    playlistNum += 1 'listnum increases by 1
End Sub

'This function enables or disbles images
Private Sub BtnshowIMG_Click(ByVal sender As System.Object, ByVal e As System.
EventArgs) Handles BtnshowIMG.Click
    If BtnshowIMG.Checked = True Then 'if checked
        MPPicBox.Visible = False 'hide picture box
        BtnshowIMG.Checked = False 'uncheck btnshowIMG
    Else
```

```
            MPPicBox.Visible = True 'show picture box
            BtnshowIMG.Checked = True 'check btnshowIMG
        End If
    End Sub
    'This function stops running the playlist
    Private Sub BtnStopPLST_Click(ByVal sender As System.Object, ByVal e As System.
    EventArgs) Handles BtnStopPLST.Click
        DeckA = False 'A is not running
        DeckB = False 'B is not running
        BtnRunA.Enabled = True 'enables loading of library track to playerA
        BtnRunB.Enabled = True 'enables loading of library track to playerB
        BtnLoadPlay.Enabled = True 'enables library load track to playlist button
        ChkCross.Checked = False 'uncheck crossfader
        ChkCross.Visible = True 'crossfader option reappears
        Try
            CmbPlayback.Enabled = True 'enabled playback selection
            BtnStopA.PerformClick() 'stop A
            BtnStopB.PerformClick() 'stop B
        Catch
            'Do nothing to prevent exception crash
        End Try
        BtnClear.Enabled = True 'btnclear enabled
        PlayTimer.Enabled = False 'playlist timer disabled
        BtnRunPlaylist.Enabled = True 'btnrunplaylist enabled
        listrun = 1 'reset list to 1
        controlreset() 'calls function to reset certain controls
        LblCDis.Visible = False 'label for crossfader deactivation is visible
        BtnStopPLST.Enabled = False 'playlist stops button disabled
        BtnAddA.Enabled = True 'enables button to add more tracks
    End Sub
    'This function loads track into PlayerA
    Private Sub loadA(ByVal trackA)
        MediaPlayerA.URL = trackA 'PlayerA gets filename from selected row
        ALoad = True 'A is ready to run
        LblTitleA.Text = MediaPlayerA.currentMedia.name 'label shows track name
        MediaPlayerA.Ctlcontrols.stop() 'stops A to get ready
        BtnPlayA.Enabled = True 'Play button for PlayerA is enabled
    End Sub
    'This function loads track into PlayerB
    Private Sub loadB(ByVal trackB)
        MediaPlayerB.URL = trackB 'PlayerB gets filename from selected row
        BLoad = True 'B is ready to run
        LblTitleB.Text = MediaPlayerB.currentMedia.name 'label shows track name
        MediaPlayerB.Ctlcontrols.stop() 'stops B to get ready
        BtnPlayB.Enabled = True 'Play button for PlayerB is enabled
    End Sub
    'This function sets up the crossfader
    Private Sub crossCheck()
        If ChkCross.Checked Then
            If DeckA Then 'if A is running...
                CrossTimer.Interval = (CDbl(CmbCross.Text) * 1000) / CInt(MediaPlayerA.
    settings.volume) 'Volume crossfade control set up
                MediaPlayerB.settings.volume = 0 'volume for B is 0
            ElseIf DeckB Then 'if B is running...
                CrossTimer.Interval = (CDbl(CmbCross.Text) * 1000) / CInt(MediaPlayerB.
    settings.volume) 'Volume crossfade control set up
                MediaPlayerA.settings.volume = 0 'volume for A is 0
            End If
        End If
    End Sub
    'This function disables certain controls when crossfading
    Private Sub KillControls(ByVal player)
        If player = "A" Then
            LblRemainA.Text = "Fading Out" 'A is fading out
            BtnBackA.Enabled = False 'rewind button for A is disabled
            BtnForwardA.Enabled = False 'fast-forward button for A is disabled
```

```
        ElseIf player = "B" Then
            LblRemainB.Text = "Fading Out" 'B is fading out
            BtnBackB.Enabled = False 'rewind button for B is disabled
            BtnForwardB.Enabled = False 'fast-forward button for B is disabled
        End If
    End Sub
    'This function monitors the running Playlist
    Private Sub PlayTimer_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles PlayTimer.Tick
        If CDAudio Then
            playCDA()
        ElseIf DeckA Or Acrossed Then
            If MediaPlayerA.Ctlcontrols.currentPosition = 0 And CrossTimer.Enabled = False ↙
    Then

                DeckA = False 'A is not running
                Acrossed = False 'A finished fading out
                If BLoad Then
                    BtnPlayB.PerformClick() 'B starts
                    ALoad = False 'A is ready for next track
                    If (CmbPlayback.Text = "Ordered" Or CmbPlayback.Text = "Continuous")    ↙
    Then
                        listrun += 1 'listrun increments by 1
                        If (list(listrun).TrackName <> "Empty") Then 'If at end of playlist↙
    ...
                            MediaPlayerA.URL = list(listrun).FileName 'A gets next track
                            LblTitleA.Text = list(listrun).TrackName 'label displays next  ↙
    track
                            ImageA = list(listrun).ImgFile 'imageA gets file
                            ALoad = True 'A is ready to run
                        Else
                            If (CmbPlayback.Text = "Continuous") Then
                                listrun = 1
                                MediaPlayerA.URL = list(listrun).FileName 'A gets next      ↙
    track
                                LblTitleA.Text = list(listrun).TrackName 'label displays    ↙
    next track
                                ImageA = list(listrun).ImgFile 'imageA gets file
                                ALoad = True 'A is ready to run
                            ElseIf (ChkCross.Checked) Then
                                ChkCross.Checked = False 'uncheck crossfader enabler
                                crossVisible() 'Calls function for crossfader visibliity
                                LblCDis.Visible = True 'label for crossfader deactivation  ↙
    is displayed
                            End If
                            BtnAddA.Enabled = False 'button to add tracks is disabled
                            BtnLoadPlay.Enabled = False 'disables adding tracks from       ↙
    library to playlist
                        End If
                    Else
                        Dim R As New Random 'random number generator
                        Dim i As Integer 'i works with R
                        i = R.Next(1, playlistNum) 'i becomes R
                        Do Until (list(i).FileName <> MediaPlayerB.URL And list(i).FileName↙
     <> "Empty" And list(i).FileName <> MediaPlayerA.URL)
                            i = R.Next(1, playlistNum) 'Gives i a new number if similar
                        Loop
                        MediaPlayerA.URL = list(i).FileName 'A gets next file
                        LblTitleA.Text = list(i).TrackName 'A gets next track
                        ImageA = list(i).ImgFile 'A gets image
                        ALoad = True 'A is ready to go
                    End If
                Else
                    BtnStopPLST.PerformClick() 'Playlist is stopped
                End If
            End If
```

```
        End If
        If DeckB Or Bcrossed Then
            If MediaPlayerB.Ctlcontrols.currentPosition = 0 And CrossTimer.Enabled = False ↙
    Then
                DeckB = False 'B is not running
                Bcrossed = False 'B finished fading out
                If ALoad Then
                    BtnPlayA.PerformClick() 'A starts
                    BLoad = False 'B is ready for next track
                    If (CmbPlayback.Text = "Ordered" Or CmbPlayback.Text = "Continuous")    ↙
    Then
                        listrun += 1 'listrun increments by 1
                        If (list(listrun).TrackName <> "Empty") Then 'If at end of playlist↙
    ...
                            MediaPlayerB.URL = list(listrun).FileName 'B gets next track
                            LblTitleB.Text = list(listrun).TrackName 'label displays next  ↙
    track
                            ImageB = list(listrun).ImgFile 'imageB gets file
                            BLoad = True 'B is ready to go
                        Else
                            If (CmbPlayback.Text = "Continuous") Then
                                listrun = 1
                                MediaPlayerB.URL = list(listrun).FileName 'A gets next      ↙
    track
                                LblTitleB.Text = list(listrun).TrackName 'label displays    ↙
    next track
                                ImageB = list(listrun).ImgFile 'imageA gets file
                                BLoad = True 'A is ready to run
                            ElseIf (ChkCross.Checked) Then
                                ChkCross.Checked = False 'uncheck crossfader enabler
                                crossVisible() 'Calls function for crossfader visibliity
                                LblCDis.Visible = True 'label for crossfader deactivation  ↙
    is displayed
                            End If
                            BtnAddA.Enabled = False 'button to add tracks is disabled
                            BtnLoadPlay.Enabled = False 'disables adding tracks from       ↙
    library to playlist
                        End If
                    Else
                        Dim R As New Random 'random number generator
                        Dim j As Integer 'j works with R
                        j = R.Next(1, playlistNum) 'j becomes R
                        Do Until (list(j).FileName <> MediaPlayerA.URL And list(j).FileName↙
     <> MediaPlayerB.URL And list(j).FileName <> "Empty")
                            j = R.Next(1, playlistNum) 'Gives j a new number if similar
                        Loop
                        MediaPlayerB.URL = list(j).FileName 'B gets next file
                        LblTitleB.Text = list(j).TrackName 'B gets next track
                        ImageB = list(j).ImgFile 'imageB gets file
                        BLoad = True 'B is ready to go
                    End If
                Else
                    BtnStopPLST.PerformClick() 'Playlist is stopped
                End If
            End If
        End If

    End Sub
    'This function initalizes PlayerA and PlayerB
    Private Sub InitializePlayers()
        MediaPlayerA.URL = "C:\MPA\init.mp3" 'A gets initialization file
        MediaPlayerB.URL = "C:\MPA\init.mp3" 'B gets initialization file
        BtnPlayA.PerformClick() 'A runs
        BtnStopA.PerformClick() 'A stops
        BtnPlayB.PerformClick() 'B runs
        BtnStopB.PerformClick() 'B stops
```

```
        LblRemainA.Text = "Ready" 'A displayed as ready
        LblRemainB.Text = "Ready" 'B displayed as ready
        ImageA = "C:\MPA\default.jpg" 'ImageA get default image
        ImageB = "C:\MPA\default.jpg" 'ImageB gets default image
    End Sub
    'This function sets the crossfader display
    Private Sub crossVisible()
        If ChkCross.Checked = True Then
            CmbCross.Visible = True 'Crossfader interval enabled
            BtnCrossNow.Visible = True 'button to crossfade enabled
            CmbCross.Text = 1 'crossfader interval set a 1
            Try
                CrossTimer.Interval = (CDbl(CmbCross.Text) * 1000) / CDbl(MediaPlayerA.
    settings.volume) 'crossafader timer set
            Catch
                CrossTimer.Interval = (1 * 1000) / 50 'crossafader timer set
            End Try
            MediaPlayerB.settings.volume = 0 'B volume at 0
            LblCDis.Visible = False 'Crossfader label disappears
        Else
            CmbCross.Visible = False 'Crossfader interval disabled
            BtnCrossNow.Visible = False 'button to crossfade disabled
        End If
    End Sub
    'This function resets the volume controls for both players
    Private Sub controlreset()
        MediaPlayerA.settings.volume = 50 'Volume for A reset
        MediaPlayerB.settings.volume = 50 'Volume for B reset
        BtnVolUpA.Visible = True 'Volume Up for A visible
        BtnVolDownA.Visible = True 'Volume Down for A visible
        BtnVolUpB.Visible = True 'Volume Up for B visible
        BtnVolDownB.Visible = True 'Volume Down for B visible
    End Sub
    'This function runs the playlist if any of the tracks are CD Audio
    Private Sub runCDA()
        Dim r As New Random
        Select Case CmbPlayback.Text 'Condition for playback
            Case Is = "Ordered"
                If list(listrun).TrackName <> "Empty" Then 'Checks if next track is empty
                    MediaPlayerA.Ctlcontrols.stop()
                    loadA(list(listrun).FileName) 'loads file from structure into playerB
                    ImageA = (list(listrun).ImgFile) 'loads image file into ImageB
                    listrun += 1 'increments listrun
                    MediaPlayerA.Ctlcontrols.play()
                    TimerA.Stop()
                Else
                    BtnStopPLST.PerformClick()
                End If
            Case Is = "Random"
                MediaPlayerA.Ctlcontrols.stop()
                Dim i As Integer 'sets up two variables from random numbers
                i = r.Next(1, playlistNum) 'i is random
                MediaPlayerA.URL = list(i).FileName 'sets file in playerA
                ImageA = list(i).ImgFile 'sets image file in playlist to imageA
                MediaPlayerA.Ctlcontrols.play()
                TimerA.Stop()
            Case Is = "Continuous"
                If list(listrun).TrackName <> "Empty" Then 'Checks if next track is empty
                    MediaPlayerA.Ctlcontrols.stop()
                    loadA(list(listrun).FileName) 'loads file from structure into playerB
                    ImageA = (list(listrun).ImgFile) 'loads image file into ImageB
                    listrun += 1 'increments listrun
                    MediaPlayerA.Ctlcontrols.play()
                    TimerA.Stop()
                Else
                    listrun = 1
```

```
                    runCDA()
                End If
        End Select
        TimerA.Start()
    End Sub
    'This function will continue to run the CD playlist
    Private Sub playCDA()
        'Playlist CDA Playback
        If (MediaPlayerA.currentMedia.duration - MediaPlayerA.Ctlcontrols.currentPosition)
    < 2 Then
            runCDA()
        End If
    End Sub
End Class
```

Source Code For The Multimedia Player's Assistant
LibraryForm.vb (Data Form)

```
Public Class LibraryForm
    Inherits System.Windows.Forms.Form
    'This form was designed using the data form wizard
    'Comments are listed where additions or modifications were made
 Windows Form Designer generated code
    'This function cancels all updates made to this record
    Private Sub btnCancel_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles btnCancel.Click
        Me.BindingContext(objLibrary2, "TrackInfo").CancelCurrentEdit()
        Me.objLibrary2_PositionChanged()

    End Sub
    'This function deletes the record
    Private Sub btnDelete_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles btnDelete.Click
        If (Me.BindingContext(objLibrary2, "TrackInfo").Count > 0) Then
            Me.BindingContext(objLibrary2, "TrackInfo").RemoveAt(Me.BindingContext
    (objLibrary2, "TrackInfo").Position)
            Me.objLibrary2_PositionChanged()
        End If

    End Sub
    'This function adds a new record to the dataset
    Private Sub btnAdd_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles btnAdd.Click
        Try
            'Clear out the current edits
            Me.BindingContext(objLibrary2, "TrackInfo").EndCurrentEdit()
            Me.BindingContext(objLibrary2, "TrackInfo").AddNew()
        Catch eEndEdit As System.Exception
            System.Windows.Forms.MessageBox.Show(eEndEdit.Message)
        End Try
        Me.objLibrary2_PositionChanged()

    End Sub
    Private Sub btnUpdate_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles btnUpdate.Click
        Try
            'Attempt to update the datasource.
            Me.UpdateDataSet()
        Catch eUpdate As System.Exception

            System.Windows.Forms.MessageBox.Show(eUpdate.Message)
        End Try
        Me.objLibrary2_PositionChanged()

    End Sub
    'This function lists the first record
    Private Sub btnNavFirst_Click(ByVal sender As System.Object, ByVal e As System.
    EventArgs) Handles btnNavFirst.Click
        Me.BindingContext(objLibrary2, "TrackInfo").Position = 0
        Me.objLibrary2_PositionChanged()

    End Sub
    'This function lists the last record
    Private Sub btnLast_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles btnLast.Click
        Me.BindingContext(objLibrary2, "TrackInfo").Position = (Me.objLibrary2.Tables(
    "TrackInfo").Rows.Count - 1)
        Me.objLibrary2_PositionChanged()

    End Sub
    'This function goes to the previous record
    Private Sub btnNavPrev_Click(ByVal sender As System.Object, ByVal e As System.
    EventArgs) Handles btnNavPrev.Click
        Me.BindingContext(objLibrary2, "TrackInfo").Position = (Me.BindingContext
```

```
(objLibrary2, "TrackInfo").Position - 1)
    Me.objLibrary2_PositionChanged()

End Sub
'This function goes to the next record
Private Sub btnNavNext_Click(ByVal sender As System.Object, ByVal e As System.
EventArgs) Handles btnNavNext.Click
    Me.BindingContext(objLibrary2, "TrackInfo").Position = (Me.BindingContext
(objLibrary2, "TrackInfo").Position + 1)
    Me.objLibrary2_PositionChanged()

End Sub
'This function notifies user whhich record this is
Private Sub objLibrary2_PositionChanged()
    Me.lblNavLocation.Text = (((Me.BindingContext(objLibrary2, "TrackInfo").Position +
1).ToString + " of  ")
                + Me.BindingContext(objLibrary2, "TrackInfo").Count.ToString)

End Sub
'This function cancels all updates to the dataset
Private Sub btnCancelAll_Click(ByVal sender As System.Object, ByVal e As System.
EventArgs) Handles btnCancelAll.Click
    Me.objLibrary2.RejectChanges()

End Sub
'This function saves all changes to the dataset
Public Sub UpdateDataSet()
    'Create a new dataset to hold the changes that have been made to the main dataset.
    Dim objDataSetChanges As MultimediaPlayer2.Library2 = New MultimediaPlayer2.
Library2
    'Stop any current edits.
    Me.BindingContext(objLibrary2, "TrackInfo").EndCurrentEdit()
    'Get the changes that have been made to the main dataset.
    objDataSetChanges = CType(objLibrary2.GetChanges, MultimediaPlayer2.Library2)
    'Check to see if any changes have been made.
    If (Not (objDataSetChanges) Is Nothing) Then
        Try
            'There are changes that need to be made, so attempt to update the
datasource by
            'calling the update method and passing the dataset and any parameters.
            Me.UpdateDataSource(objDataSetChanges)
            objLibrary2.Merge(objDataSetChanges)
            objLibrary2.AcceptChanges()
        Catch eUpdate As System.Exception

            Throw eUpdate
        End Try

    End If

End Sub
Public Sub LoadDataSet()
    'Create a new dataset to hold the records returned from the call to FillDataSet.
    'A temporary dataset is used because filling the existing dataset would
    'require the databindings to be rebound.
    Dim objDataSetTemp As MultimediaPlayer2.Library2
    objDataSetTemp = New MultimediaPlayer2.Library2
    Try
        'Attempt to fill the temporary dataset.
        Me.FillDataSet(objDataSetTemp)
    Catch eFillDataSet As System.Exception
        'Add your error handling code here.
        Throw eFillDataSet
    End Try
    Try
        'Empty the old records from the dataset.
```

```
            objLibrary2.Clear()
            'Merge the records into the main dataset.
            objLibrary2.Merge(objDataSetTemp)
        Catch eLoadMerge As System.Exception
            'Add your error handling code here.
            Throw eLoadMerge
        End Try

    End Sub
    Public Sub UpdateDataSource(ByVal ChangedRows As MultimediaPlayer2.Library2)
        Try
            'The data source only needs to be updated if there are changes pending.
            If (Not (ChangedRows) Is Nothing) Then
                'Open the connection.
                Me.OleDbConnection1.Open()
                'Attempt to update the data source.
                OleDbDataAdapter1.Update(ChangedRows)
            End If
        Catch updateException As System.Exception

            Throw updateException
        Finally
            'Close the connection whether or not the exception was thrown.
            Me.OleDbConnection1.Close()
        End Try

    End Sub
    Public Sub FillDataSet(ByVal dataSet As MultimediaPlayer2.Library2)
        'Turn off constraint checking before the dataset is filled.
        'This allows the adapters to fill the dataset without concern
        'for dependencies between the tables.
        dataSet.EnforceConstraints = False
        Try
            'Open the connection.
            Me.OleDbConnection1.Open()
            'Attempt to fill the dataset through the OleDbDataAdapter1.
            Me.OleDbDataAdapter1.Fill(dataSet)
        Catch fillException As System.Exception

            Throw fillException
        Finally
            'Turn constraint checking back on.
            dataSet.EnforceConstraints = True
            'Close the connection whether or not the exception was thrown.
            Me.OleDbConnection1.Close()
        End Try

    End Sub
    'This function loads the dataset
    Private Sub LibraryForm_Load(ByVal sender As System.Object, ByVal e As System.
    EventArgs) Handles MyBase.Load
        Try
            'Attempt to load the dataset.
            Me.LoadDataSet()
            Dim dm As CurrencyManager = _
            CType(BindingContext _
            (objLibrary2, "TrackInfo"), CurrencyManager)
            Dim dv As DataView = CType(dm.List, DataView)
            dv.Sort = "TrackName"
        Catch eLoad As System.Exception
            'Add your error handling code here.
            'Display error message, if any.
            System.Windows.Forms.MessageBox.Show(eLoad.Message)
        End Try
        Me.objLibrary2_PositionChanged()
    End Sub
```

```
    'This function opens the file dialog for media opening
    Private Sub BtnFile1_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) ↙
    Handles BtnFile1.Click
        OpenFileDialog1.Filter = "Audio Files | *.mp3;*.wav;*.snd;*.au;*.aif;*.aifc;*.aiff;↙
    *.wma;*.mid;*.rmi;*.midi" & _
        "|Video Files | *.avi;*.mpeg;*.mpg;*.m1v;*.mp2;*.mpa;*.mpe;*.ifo;*.vob;*.wmv|CD T" ↙
    & _
        "racks|*.cda"
        OpenFileDialog1.ShowDialog()
        If OpenFileDialog1.FileName.EndsWith("cda") Then
            MsgBox("CD Audio Track Cannot be Added to Library", MsgBoxStyle.Information,   ↙
    "NO CD Audio")

        Else
            editFileName.Text = OpenFileDialog1.FileName
            If ChkPrev.Checked Then
                Previewer.URL = editFileName.Text
                runprev()
            End If
        End If

    End Sub
    'This function allows user to preview the track
    Private Sub ChkPrev_CheckedChanged(ByVal sender As System.Object, ByVal e As System.   ↙
    EventArgs) Handles ChkPrev.CheckedChanged
        If ChkPrev.Checked Then
            Try
                If editFileName.Text <> "" Then
                    Previewer.URL = editFileName.Text
                    runprev()
                    Timer1.Start()
                Else
                    BtnPreview.Visible = False
                    Previewer.Ctlcontrols.stop()
                End If
            Catch
                MsgBox("No file or invalid file selected", MsgBoxStyle.Critical, "Error")
            End Try
        Else
                BtnPreview.Text = "Stop"
                BtnPreview.Visible = False
                Previewer.Ctlcontrols.stop()
                Timer1.Stop()
        End If

    End Sub
    'This function previews the track
    Private Sub BtnPreview_Click(ByVal sender As System.Object, ByVal e As System.         ↙
    EventArgs) Handles BtnPreview.Click
        If BtnPreview.Text = "Stop" Then
            Previewer.Ctlcontrols.stop()
            BtnPreview.Text = "Play"
        Else
            Previewer.Ctlcontrols.play()
            BtnPreview.Text = "Stop"
        End If
    End Sub
    'This function updates the track name and duration
    Private Sub runprev()
        BtnPreview.Visible = True
        If editTrackName.Text = "" Then
            Try
                editTrackName.Text = Previewer.currentMedia.name()
            Catch
                ChkPrev.Checked = False
            End Try
```

```
        End If
        Try
            editDuration.Text = Previewer.currentMedia.durationString
        Catch
            ChkPrev.Checked = False
        End Try
    End Sub
    'This function opens the file dialog for image files
    Private Sub BtnFile2_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles BtnFile2.Click
        OpenFileDialog1.Filter = "Image Files | *.bmp;*.jpg;*.gif"
        OpenFileDialog1.ShowDialog()
        editTrackPic.Text = OpenFileDialog1.FileName
    End Sub
    'This function updates the duration when track is running
    Private Sub Timer1_Tick(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Handles Timer1.Tick
        runprev()
        Timer1.Stop()
    End Sub
    'This function searches for the track name
    'Source code provided by Visual Basic.NET in Easy Steps
    'by Tim Anderson
    Private Sub btnSearch_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
     Handles btnSearch.Click
        Dim searchName As String
        Dim i As Integer
        searchName = InputBox("Enter Track Name:", "Search For Track")
        If searchName <> "" Then
            Dim dm As CurrencyManager = _
            CType(BindingContext _
            (objLibrary2, "TrackInfo"), CurrencyManager)
            Dim dv As DataView = CType(dm.List, DataView)
            i = dv.Find(searchName)
            If i > -1 Then
                Me.BindingContext _
                (objLibrary2, "TrackInfo").Position = i
                Me.objLibrary2_PositionChanged()
            End If
        End If
    End Sub
End Class
```

Data Table For Library

SQL For Data Table

```
CREATE TABLE TrackInfo
(       TrackNumber COUNTER,
        TrackName   Text(50),
        Artist      Text(25),
        Duration    Text(8),
        Genre       Text(10),
        TrackPic    Text(50),
Constraint PKTrackNumber Primary Key (TrackNumber))
```

SQL For Query Dataset

```
SELECT TrackNumber, TrackName, Artist, Duration, Genre,
FROM TrackInfo
ORDER BY TrackNumber
```

The Multimedia Player's Assistant

User's Manual

To install The Multimedia Player's Assistant, insert disc into CD/DVD Drive. The installation program should automatically begin. If not, click on:

Start → Run → Type "D:\setup.exe" in prompt (Use drive label for D) and click 'OK.'

Make sure the setup utility installs the program in the directory named, "C:\MPA".

Once installed, click on:

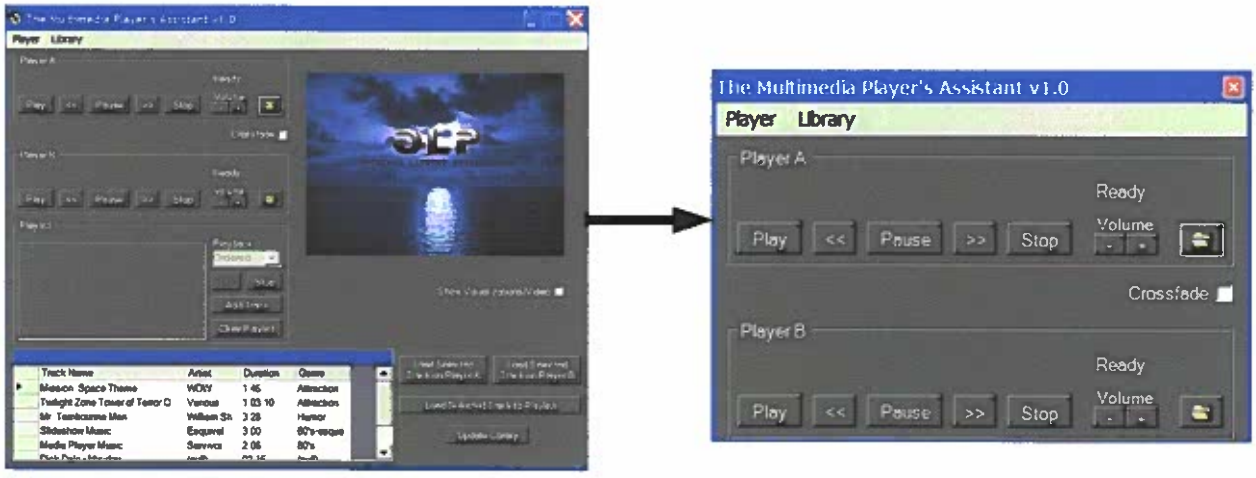
Start → (All) Programs → The Multimedia Player's Assistant

A display similar to above should be displayed.

To expand the display to include the Track Library, click on:

Library → Show Library

Compact Display



Normal View With Library

Compact View

Switching to Compact View will reduce the size of the display to ¼. To do this, close the library:

Library → Show Library (To Close)

Then, click on:

Player → Compact View

To return to normal view, click on:

Player → Normal View

Basic Player Functions

Folder Icon—Opens a file to run on the player.

Play—Plays the currently loaded file/track.

<< -- Rewinds the track by 10 seconds

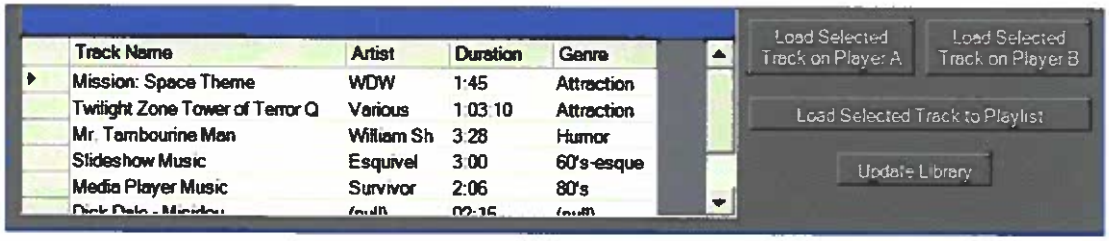
Pause—Pauses the track.

>> -- Fast Forwards the track by 10 seconds.

Stop—Stops the track.

Volume—The "-" button reduces the volume and the "+" button increases the volume.

Track Library



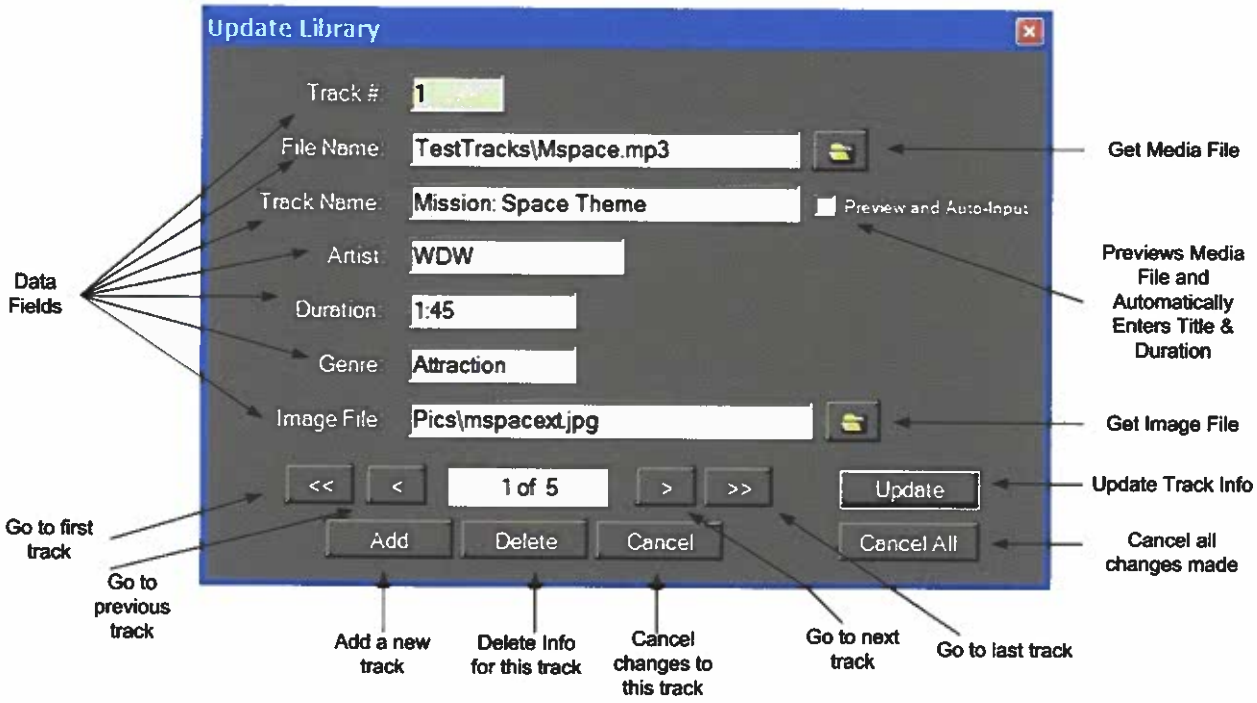
The Track Library allows you to add, remove, and play any audio or video track on this data set. Click on any row in the data set to select the track. You have three options with the selected track:

1. "Load Selected Track on Player A"- Loads selected track on Player A
2. "Load Selected Track on Player B"- Loads selected track on Player B
3. "Load Selected Track to Playlist"- Adds the selected track to the playlist.

To add, delete, or modify tracks on the data set, click on "Update Library".

(See Page titled Library Form for more information on track adding, deleting, or modifying)

Library Form



When "Preview and Auto-Input" is checked, the file in the File Name box will be loaded and playing, while automatically inputting the name of the track and its duration to the form. You must enter all other fields except for 'Track #'. Additionally, a "Play" or "Stop" button will appear when previewing the track.

When finished, click on "Update" before closing.

NOTE: BECAUSE CD AUDIO TRACKS ARE REMOVABLE, THEY CANNOT BE ADDED TO THE LIBRARY.

Playlist Controls

You can either add tracks from the library or by clicking on, "Add Track" and selecting a file. Each track is assigned a number.

There are 3 options for playlist playback:

1. Ordered—All tracks in playlist are run in order and stops after the last track is played.
2. Random—Each track from the playlist is randomly selected to be played.
3. Continuous—Same as 'Ordered', but starts over when the last track is loaded.

Each track in the playlist is loaded alternately into Players A and B and rotate playback for each track. The Crossfader is automatically activated when the playlist is run by clicking the "Run" button. To stop the playlist, click on "Stop". To clear and reset the playlist, click on "Clear Playlist".

If you don't want to use the crossfader when running a playlist, leave the playlist number set at "1".

NOTE: CD AUDIO TRACKS CAN BE ADDED. HOWEVER, GIVEN THE NATURE OF CD AUDIO DISCS, ALL TRACKS ARE RUN ON PLAYER A ONLY AND THE CROSSFADER IS DISABLED.

The Crossfader

The Crossfader is a track-transitioning tool that fades out the volume of one track and fades in another. The number box next to the "Crossfade Now" button sets the length of the transition from 1 to 5 to 10. When the track running reaches the sets amount of time left, the crossfader is activated. To bypass waiting until the end of the track, you click on "Crossfade Now". If a playlist is running and the last track is currently playing (In Ordered Playback), the crossfader is disabled.

Images, Visualizations, & Video

Images that are added to the library for each track are displayed in the box to the right of the player controls. If no image was selected, the default image is displayed.

To activate visualizations for each track or if you plan to run video, click on the "Show Visualizations/Video" button.

To hide or show images, click on:

Library → Show Images

Resources Used

•MSDN Online Library
–Windows Media Player Active Component Implementation

•Visual Basic.NET in easy steps (Programming for Windows and the Web) by Tim Anderson
–Data Table & Form Implementation

•An Introduction to Programming Using Visual Basic.NET (5th Edition) by David L. Schneider
–Other Functions and coding